שאלה 1

בשאלה זו אין להשתמש בפונקציות מהספרייה הסטנדרטית למעט malloc ו-free. אין צורך לבדוק הצלחה של הקצאת זכרון. הפונקציות צריכות להחזיר ערכים מתאימים (ולא לצאת עם exit במקרה של שגיאה) עבור כל פרמטרים שיועברו אליהן.

א. ממשו פונקציה בשם is_substring שמקבלת שתי מחרוזות s1 ו-s2 ומחזירה את האינדקס של המופע הראשון של s2 ב-s1 או 1- אם s2 לא מופיעה ב-s1. יש להבדיל בין אותיות uppercase ו-lowercase (כלומר: foo לא נחשבת תת מחרוזת של Food).

למשל:

עבור s1="abracadabra" ו-s2="cada" הפונקציה תחזיר 4

עבור s1=s2="abracadabra" הפונקציה תחזיר 0

עבור s1="abracadabra" ו-s2="bara" הפונקציה תחזיר 1-

```
int is_substring(const char* str, const char* substr) {

    if (str == NULL || substr == NULL)
        return -1;

    if (*substr == '\0')
        return 0; // An empty substring is always found

    for(const char* str2 = str; *str2 != '\0'; str2++) {
        if(*str2 != *substr)
            continue;

        const char *s, *sub;
        for(s = str2, sub = substr; *s && *sub && *s == *sub; s++, sub++)
            ;

        if(*sub == '\0') {
            return str2-str; // Found the substring
        }
    }

    return -1; // Substring not found
}
```

ב. נגדיר rotation של מחרוזת כהזזה מעגלית של תווי המחרוזת תוך שמירה על הסדר.

למשל:

המחרוזת CDAB היא rotation של המחרוזת ABCD (הזזנו את התווים 2 מקומות קדימה)

GPTChat היא rotation של ChatGPT

elloh היא rotation של hello

Roxanne היא rotation של Roxanne

dofo **אינה** rotation של food מכיוון שסדר האותיות שונה

ממשו פונקציה בשם is_rotation שמקבלת שתי מחרוזות s1 ו-s2 ומחזירה true אם s2 היא rotation של s1 או false אחרת.

```
// The idea: duplicate s1 and check if s2 is a substring of the duplicated string.
// If it is, then s2 is a rotation of s1.
bool is_rotation(const char* s1, const char* s2) {
    if (s1 == NULL || s2 == NULL)
        return false;

    unsigned int len1 = 0;
    while(s1[len1] != '\0') len1++;

    char* s1s1 = (char*)malloc((2 * len1 + 1) * sizeof(char));
    if(s1s1 == NULL)
        return false; // Memory allocation failed

    for(int i = 0; i < len1; i++) {
        s1s1[i] = s1s1[i + len1] = s1[i];
    }
    s1s1[2 * len1] = '\0';

    bool flag = is_substring(s1s1, s2) != -1;
    free(s1s1);
    return flag;
}
```

שאלה 2

נתון הקוד הבא. ענו על השאלות הבאות:

א. מה ידפיס הקוד?

ב. מה עושה הפונקציה mystery? הסבירו את הפעולה שהיא מבצעת (למשל: "הפונקציה מחשבת סכום על מערך מספרים"). תשובות שמכילות פירוט של הקוד לא תתקבלנה (למשל: "הפונקציה מריצה לולאה על המערך ועבור כל i מוסיפה את הערך במקום ה-i למשתנה sum. בסוף הפונקציה מחזירה את sum").

ג. אילו בעיות קיימות או עשויות להתעורר בהרצת הפונקציה ()mystery ובעקבותיה ()recursive_mystery אם נשנה את ()main? תארו לפחות 5 בעיות וציינו מאיזה סוג הן (שגיאת זמן ריצה, שגיאה לוגית – הפונקציה לא תעבוד כמו שצריך, וכו).

למשל:

שורה 28, בקריאה ל-mystery: אם הערך ב-nletters יהיה שונה מאורך המערך letters עשויה לקרות שגיאת זמן ריצה (אם nletters גדול מאורך המערך) או שגיאה לוגית (אם קטן).

אין להשתמש בדוגמא הזאת כחלק מהתשובה.

```c
#include <stdio.h>
#include <stdlib.h>

/*****************************************************************************/
void recursive_mystery(char* letters, int nletters, char* arr, int index, int n) {
    if(index == n) {
        arr[index] = '\0';
        printf("%s\n", arr);
        return;
    }
    for(int i = 0; i < nletters; i++) {
        arr[index] = letters[i];
        recursive_mystery(letters, nletters, arr, index + 1, n);
    }
}

/*****************************************************************************/
void mystery(char* letters, int nletters, int n) {
    char* arr = (char*)malloc((n+1)*sizeof(char));
    recursive_mystery(letters, nletters, arr, 0, n);
}

/*****************************************************************************/
int main() {
    char letters[] = {'A', 'B', 'C'};
    int nletters = sizeof(letters) / sizeof(letters[0]);
    int n = 2;
    mystery(letters, nletters, n);
    return 0;
}
```

א.

AA
AB
AC
BA
BB
BC
CA
CB
CC

ב. הפונקציה מדפיסה את כל הצירופים האפשריים באורך n של האותיות שהועברו.

ג. 1. אין שחרור זכרון ל-malloc – דליפת זכרון
2. אין בדיקה ש-malloc הצליחה – אם לא אז תהיה שגיאת זמן ריצה
3. אם letters הוא NULL – שגיאת זמן ריצה בגלל שאין בדיקה שהכתובת חוקית
4. אם nletters אינו האורך של letters – או שלא יודפסו כל הצירופים (התוכנית תרוץ אבל לא תעבוד כמו שצריך) או שתהיה חריגה מהזכרון של letters (שגיאת זמן ריצה)

שאלה 3

שאלה זו עוסקת בעצי חיפוש בינארים. במימוש שלנו תת העץ השמאלי מכיל ערכים שקטנים מ- או שווים ל-data ותת העץ הימני מכיל ערכים שגדולים מ-data. בשני הסעיפים מותר להשתמש בהגדרות הבאות:

```
typedef struct Node* Node;

// A node in a binary search tree
struct Node {
    int data;
    Node left, right;
};

// Function to create a new node
Node NodeCreate(int data) {
    Node newNode = (Node)malloc(sizeof(struct Node));
    if(!newNode)
        return NULL;
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

א. ממשו את הפונקציה BSTClone שמקבלת את השורש של עץ חיפוש בינארי שמכיל מספרים שלמים ומחזירה עותק חדש של העץ כולו.

```
Node BSTClone(Node root) {
    if (root == NULL)
        return NULL;

    Node newNode = NodeCreate(root->data);
    newNode->right = BSTClone(root->right);
    newNode->left = BSTClone(root->left);
    return newNode;
}
```

ב. ממשו את הפונקציה BSTCloneGT שמקבלת שורש של עץ חיפוש בינארי וכן מספר שלם x ומחזירה עץ חיפוש בינארי שבו רק הצמתים שבהם data גדול מ-x נמצאים. אין לממש פונקציות עזר או

פעולות אחרות של עץ חיפוש בינארי שלמדנו בכתה. מותר להשתמש ב-createNode וכמובן בהגדרות של Node ו-struct Node הנתונות.

```
Node BSTCloneGT(Node root, int x) {
    if (root == NULL)
        return NULL;

    if (root->data > x) {
        Node newNode = NodeCreate(root->data);
        newNode->right = BSTCloneGT(root->right, x);
        newNode->left = BSTCloneGT(root->left, x);
        return newNode;
    }

    return BSTCloneGT(root->right, x);
}
```

שאלה 4

נרצה לממש מודול שמייצג משחקים בליגת העל בכדורגל. המודול ישמור את המידע הבא:

- שתי הקבוצות המשחקות. לכל קבוצה בליגה ינתן מספר מזהה. בליגת העל יש 14 קבוצות, מזהי הקבוצות יהיו בין 0-13.

- מספר השערים שכל קבוצה הבקיעה. המספר הכי גדול של שערים שהובקע אי פעם בליגה על ידי קבוצה אחת הוא 13, נניח שמספר השערים האפשרי הוא בין 0 ל-15.

נתון הממשק של המודול, עם setters ו-getters לכל המידע.

לפני פירוט המשימה, שימו לב לנקודות הבאות:

א. המודול צריך להיות חסכוני במקום ככל האפשר. מימוש לא חסכוני יגרור הורדת נקודות משמעותית.

ב. המנעו משכפול קוד על ידי שימוש בפונקציות קיימות. ניתן להשתמש בכל פונקציה שמוגדרת בממשק גם אם לא מימשתם אותה.

ג. אין צורך לבדוק הצלחת הקצאה דינמית או נכונות פרמטרים שמועברים לפונקציות

המשימה:

מתוך המודול Game ממשו את הפרטים הבאים בלבד.

- המבנה struct Game. פרטו בהערה איך המידע נשמר ב-struct.

- הפונקציה GameCreate שיוצרת אובייקט Game חדש

- הפונקציה GameGetTeam1 שמחזירה את הקבוצה הראשונה

- הפונקציה GameSetTeam2 שמשנה את הקבוצה השניה

- הפונקציה GameGetGoals1 שמחזירה את מספר השערים שהבקיעה הקבוצה הראשונה

- הפונקציה GameSetGoals2 שמשנה את מספר השערים שהבקיעה הקבוצה השניה

```c
#ifndef GAME_H
#define GAME_H

typedef struct Game* Game;
typedef unsigned char TeamID;
typedef unsigned char Goals;

/*
 * Creates a new Game instance with the given parameters.
 * team1 and team2 are the IDs of the two teams. Can be in the range [0, 14]
 * goals1 and goals2 are the number of goals scored by each team. Can be in the range [0, 15]
 */
Game GameCreate(TeamID team1, TeamID team2, Goals goals1, Goals goals2);

void GameDestroy(Game game);

TeamID GameGetTeam1(Game game);
void GameSetTeam1(Game game, TeamID team1);

TeamID GameGetTeam2(Game game);
void GameSetTeam2(Game game, TeamID team2);

Goals GameGetGoals1(Game game);
void GameSetGoals1(Game game, Goals goals1);

Goals GameGetGoals2(Game game);
void GameSetGoals2(Game game, Goals goals2);

#endif // GAME_H
```

```c
#include "Game.h"
#include <stdlib.h>

// bits: 0-3 team1 (0-14)
//       4-7 team2 (0-14)
//       8-11 goals1 (0-15)
//       12-15 goals2 (0-15)

struct Game {
    unsigned short data;
};

Game GameCreate(TeamID team1, TeamID team2, Goals goals1, Goals goals2) {
    Game newGame = (Game)malloc(sizeof(struct Game));
    if (!newGame) {
        return NULL;
    }
    GameSetTeam1(newGame, team1);
    GameSetTeam2(newGame, team2);
    GameSetGoals1(newGame, goals1);
    GameSetGoals2(newGame, goals2);

    return newGame;
}

void GameDestroy(Game game) {
    free(game);
}

TeamID GameGetTeam1(Game game) {
    return (game->data >> 0) & 0xF;
}

void GameSetTeam1(Game game, TeamID team1) {
    game->data &= ~(0x000F << 0);
    game->data |= (team1 & 0xF) << 0;
}

TeamID GameGetTeam2(Game game) {
    return (game->data >> 4) & 0xF;
}

void GameSetTeam2(Game game, TeamID team2) {
    game->data &= ~(0x000F << 4);
    game->data |= (team2 & 0x000F) << 4;
}

Goals GameGetGoals1(Game game) {
    return (game->data >> 8) & 0xF;
}

void GameSetGoals1(Game game, Goals goals1) {
    game->data &= ~(0x000F << 8);
    game->data |= (goals1 & 0x000F) << 8;
}

Goals GameGetGoals2(Game game) {
    return (game->data >> 12) & 0xF;
}

void GameSetGoals2(Game game, Goals goals2) {
    game->data &= ~(0x000F << 12);
    game->data |= (goals2 & 0x000F) << 12;
}
```

שאלה 5

מבנה הנתונים Priority Queue שומר נתונים בהתאם לקריטריון קדימות שמוגדר על ידי המשתמשת. למשל, עבור priority queue ששומר מילים ושקריטריון הקדימות שלו הוא סדר מילוני, ימי השבוע ישמרו בסדר הבא (משמאל לימין) ללא קשר לסדר ההוספה:

Friday, Monday, Saturday, Sunday, Thursday, Tuesday, Wednesday

בשאלה זו נממש Priority Queue בעזרת linked list. במימוש שלנו אובייקט Priority Queue ייצור עותק חדש של כל אלמנט שיוכנס אליו.

א. כתבו את הקובץ PQueue.h שמכיל ממשק ל-Priority queue גנרי ומאפשר את הפעולות הבאות:

- PQueueCreate – יוצרת Priority Queue חדש ומחזירה אותו
- PQueueDestroy – משחררת Priority Queue
- PQueuePush – מוסיפה אלמנט ל-Priority Queue, לא מחזירה דבר
- PQueuePop – מוציאה ומחזירה את האלמנט שבראש התור (שהוא האלמנט עם העדיפות הכי גבוהה)
- PQueueSize – מחזירה את מספר האלמנטים בתור

```
#ifndef PQUEUE_H
#define PQUEUE_H

#include "LinkedList.h"

typedef struct PQueue_t* PQueue;

PQueue PQueueCreate(Element (*copyElement)(Element), void (*freeElement)(Element), int
(*compareElements)(Element, Element));
void PQueueDestroy(PQueue pq);
void PQueuePush (PQueue pq, Element element);
Element PQueuePop (PQueue pq);
unsigned int PQueueSize (PQueue pq);

#endif // PQUEUE_H
```

ב. ממשו את struct PQueue ואת PQueueCreate. שמירת המידע ב-PQueue חייבת להעשות בעזרת LinkedList שהממשק שלו נתון כאן.

```
#ifndef LINKEDLIST_h
#define LINKEDLIST_h

/* A linked list of elements.
   The list stores the addresses of the elements passed to it and does not create new copies.
   When destroying the list, the elements are not freed.
*/

typedef void* Element;
typedef struct LinkedList* LinkedList;

/* Return an empty list */
LinkedList      LLCreate();

/* Destroys a list without freeing the elements */
void            LLDestroy(LinkedList ll);

/* Returns the number of elements in a list */
unsigned int    LLSize(LinkedList ll);

/* Adds an element at location index (between 0 to LLSize()). index=0 means that the
   element will be added to the beginning of the list, index=1 will place the
   element between the first two items, and index=LLSize() will place the
   element at the end of the list. */
void            LLAdd(LinkedList ll, unsigned int index, Element element);

/* Removes the element from the list and returns it(index between 0 to LLSize-1) */
Element         LLRemove(LinkedList ll, unsigned int index);

/* Retuens the element at index without removing it  */
Element         LLGet(LinkedList ll, unsigned int index);

#endif
```

```
struct PQueue_t {
    LinkedList list;
    Element (*copyElement)(Element);
    void (*freeElement)(Element);
    int (*compareElements)(Element, Element);
};

PQueue PQueueCreate(Element (*copyElement)(Element), void (*freeElement)(Element),
                    int (*compareElements)(Element, Element)) {
    PQueue pq = (PQueue)malloc(sizeof(struct PQueue_t));
    if (!pq) {
        return NULL;
    }
    pq->list = LLCreate();
    if (!pq->list) {
        free(pq);
        return NULL;
    }
    pq->copyElement = copyElement;
    pq->freeElement = freeElement;
    pq->compareElements = compareElements;

    return pq;
}
```

ג. ממשו את PQueuePush

```
void PQueuePush (PQueue pq, Element element) {
    Element elementCopy = pq->copyElement(element);
    if (!elementCopy) {
        return;
    }

    unsigned int size = LLSize(pq->list);
    unsigned int index = 0;
    for (index = 0; index < size &&
         pq->compareElements(elementCopy, LLGet(pq->list, index)) > 0; index++)
        ;

    LLAdd(pq->list, index, elementCopy);
}
```